HEAVEN'S LIGHT IS OUR GUIDE

DEPARTMENT OF MECHATRONICS ENGINEERING
RAJSHAHI UNIVERSITY OF ENGINEERING & TECHNOLOGY

Project Report

CSE-1288

Project Name : Sudoku Game developed using C programming language

Submitted To :

Md. Manirul Islam
Assistant Professor
Dept. of Mechatronics Engineering
RUET

Md. Mehedi Hasan
Lecturer
Dept. of Mechatronics Engineering
RUET

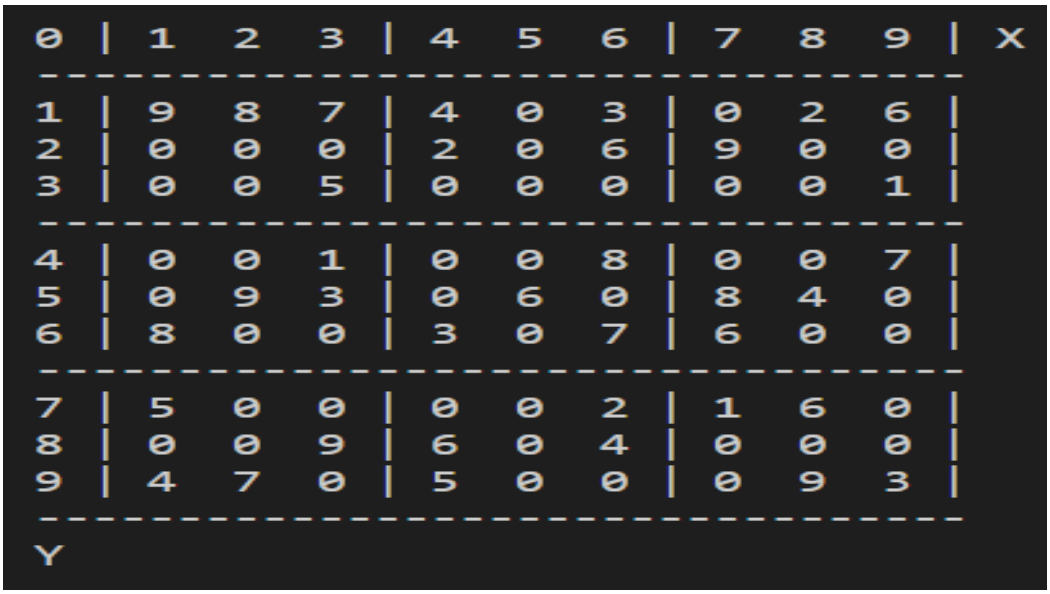
Submitted By : Fahim Faisal
ID - 2008001

Introduction

Sudoku is an interesting game where the player has to solve a puzzle by completing numbers 1-9 in a row or in a set of 3x3 square tiles which form a bigger 3x3 set. Therefore a bigger 9x9 overall set of numbers will form where each rows and column will also contain numbers 1-9. The puzzle contains many blank cells which players will be given to fill up. Players can guess numbers by observing the whole puzzle.

This sudoku game is fully written c programming language. In this game players will be presented a sudoku puzzle in the console. Players will guess possible numbers to fill up blank cells which are marked by ‘ 0 ’. Players will need to fill all the blank cells to solve the sudoku puzzle.

Players will be given a puzzle like the one in the image below



This game has been developed using the C programming language with three header files. This application is written with 8 functions and with dynamic memory allocation. The main function just calls the defined functions and frees up dynamic memories allocated by the malloc() functions in the program in the end.

How To Play

In this game players will be asked to which cell they want to input a number which they have guessed. To specify the cell, players will have to input the XY coordinate of the cell they wan to input a number for. Afterwards the players will be able to input the number they have guessed for that particular cell. If the number is correct then players will be able to proceed further and a new puzzle with that cell filled with the correct number that was guessed by players. If the guessed number was incorrect then players will be shown a message which will let them know about their incorrect guess. Players will be able to continue likewise until the whole puzzle is solved.
Below is an example of different inputs asked by the program but with an incorret value.

```
Press Enter to continue.


 Enter Coordinate  in XY format :
5 1

Please insert value from 1 to 9
4

incorrect value for the X = 5 Y = 1 coordinate, please try again
```

If the player inputs a correct value for the cell then the value will be put into the puzzle and a new puzzle with the cell filled with the correct value will be displayed in the console like the example below.

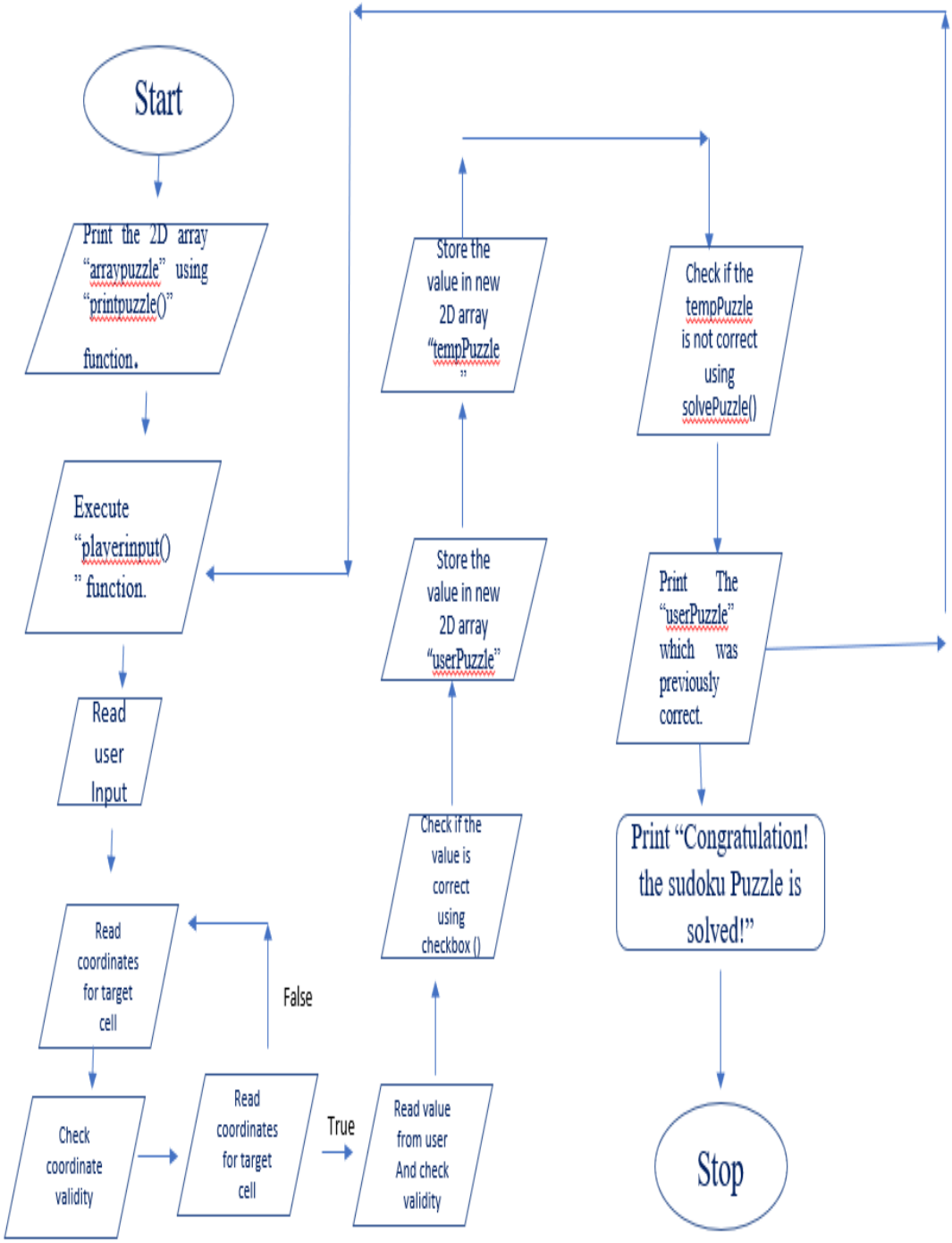
```
 Enter Coordinate  in 'X Y' format :
5 1

Please insert value from 1 to 9
1

 0 | 1  2  3 | 4  5  6 | 7  8  9 | X
 ---------------------------------
 1 | 9  8  7 | 4  1  3 | 0  2  6 |
 2 | 0  0  0 | 2  0  6 | 9  0  0 |
 3 | 0  0  5 | 0  0  0 | 0  0  1 |
 ---------------------------------
 4 | 0  0  1 | 0  0  8 | 0  0  7 |
 5 | 0  9  3 | 0  6  0 | 8  4  0 |
 6 | 8  0  0 | 3  0  7 | 6  0  0 |
 ---------------------------------
 7 | 5  0  0 | 0  0  2 | 1  6  0 |
 8 | 0  0  9 | 6  0  4 | 0  0  0 |
 9 | 4  7  0 | 5  0  0 | 0  9  3 |
 ---------------------------------
 Y
```

In the example above we can see that a correct value has been inserted by the player after which The cell with the coordinate ‘5 1’ has been displayed with the inserted value ‘1’.

Algorithm

Step 1 : Start
Step 2 : Print the 2D array “arraypuzzle” using “printpuzzle()” function.
Step 3: Execute “playerinput()” function.
Step 4: If pressed enter proceed to step 4. But if pressed ‘q’ then goto step 17.
Step 5: Ask user for coordinate of the target cell for entering guessed number in “XY” format.
Step 6: Check if the entered coordinate is valid using condition ((positionX > 9) || (positionX < 1) || (positionY > 9) || (positionY < 1)
|| (userPuzzle[positionY - 1][positionX - 1] != 0))
Step 7: If coordinates are not valid read for coordinates again and goto step 5.
Step 8: If coordinates are valid , then read value for the specific cell of the coordinates.
Step 9: If the value satisfies ((userVal > 9) || (userVal < 1)) then print "please insert valid value:” and read uservalue again.
Step 10: Check valid values if they are correct using “checkbox()” function.
Step 11: If the value is correct ,then store the value in a new 2D array puzzle called “userPuzzle”.
Step 12: Store the puzzle with values given by the player in a new 2D array called “tempPuzzle”.
Step 13: Agin check if “tempPuzzle” is incorrect using “solvePuzzle()”. If incorrect then again assign 0 to the user inputted
coordinate.
Step 14:Print The “userPuzzle” which was previously correct.
Step 15: Goto step 3. But if there are no available blank cells i.e, all the blank cells are filled up by correct inputs , then goto step
16.
Step 16: Print “Congratulation! the sudoku Puzzle is solved!”
Step 17: Stop

Flowchart


Start
Print the 2D array "arraypuzzle" using "printpuzzle()" function.
Execute "playerinput() " function.
Read user Input
Read coordinates for target cell
Check coordinate validity
False
Read coordinates for target cell
True
Read value from user And check validity
Check if the value is correct using checkbox ()
Store the value in new 2D array "userPuzzle"
Store the value in new 2D array "tempPuzzle "
Check if the tempPuzzle is not correct using solvePuzzle()
Print The "userPuzzle" which was previously correct.
Print "Congratulation! the sudoku Puzzle is solved!"
Stop

C Code :

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>


int **createPuzzle();
void printPuzzle(int **puzzle);
bool checkAvailable(int **puzzle, int *row, int
*column);
bool checkBox(int **puzzle, int row, int column, int
val);
bool solvePuzzle(int **puzzle);
int **copyPuzzle(int **puzzle);
void userChoice(int **userPuzzle, int **tempPuzzle);

int main()
{
    int** puzzle = createPuzzle();
    int** userPuzzle = copyPuzzle(puzzle);
    int** tempPuzzle = copyPuzzle(puzzle);
    printf("       PLAY SUDOKU GAME:\n\n");
    printf("      Solve the sudoku puzzle below
\n\n");

    printPuzzle(userPuzzle);
    userChoice(userPuzzle,tempPuzzle);
    free(puzzle);
    free(userPuzzle);
    free(tempPuzzle);
}
int** createPuzzle(){
    int i, j;
    int** puzzle;
    int arrayPuzzle[9][9] = { 9, 8, 7,   4, 0,
3,   0, 2, 6,
                              0, 0, 0,   2, 0,
6,   9, 0, 0,
                              0, 0, 5,   0, 0,
0,   0, 0, 1,

                              0, 0, 1,   0, 0,
8,   0, 0, 7,
                              0, 9, 3,   0, 6,
0,   8, 4, 0,
                              8, 0, 0,   3, 0,
7,   6, 0, 0,

                              5, 0, 0,   0, 0,
2,   1, 6, 0,
                              0, 0, 9,   6, 0,
4,   0, 0, 0,
                              4, 7, 0,   5, 0,
0,   0, 9, 3, };

    puzzle = (int**)malloc(sizeof(int*) * 9);

 for (i = 0; i < 9; i++){
        puzzle[i] = (int*)malloc(sizeof(int) * 9);
        for(j = 0; j < 9; j++){
            puzzle[i][j] = arrayPuzzle[i][j];
        }
    }
    return puzzle;
}

bool checkBox(int** puzzle, int row, int
column, int val){
    int i, j;
    int squareRow, squareColumn;
 //CHECK SQUARE
    squareRow = row - row % 3;
    squareColumn = column - column % 3;

    for(i = 0; i < 3; i++){
        for(j = 0; j < 3; j++){
            if(puzzle[squareRow +
i][squareColumn + j] == val)
                return false;
        }
    }

    return true;
}


int** copyPuzzle(int** puzzle){
    int i, j;
    int** newPuzzle;

    newPuzzle = (int**)malloc(sizeof(int*) *
9);
    for (i = 0; i < 9; i++){
        newPuzzle[i] =
(int*)malloc(sizeof(int) * 9);
        for(j = 0; j < 9; j++){
            newPuzzle[i][j] = puzzle[i][j];
        }
    }
    return newPuzzle;
}



bool solvePuzzle(int** puzzle){
    int i, j, val;

    if(!checkAvailable(puzzle, &i, &j))
        return true;

    for(val = 1; val < 10; val++){
        if(checkBox(puzzle, i, j, val)){
            puzzle[i][j] = val;
            if(solvePuzzle(puzzle))
                return true;
            else
                puzzle[i][j] = 0;
        }
    }
    return false;
}
```

```
vod printPuzzle(
    int i, j, a;

    printf("\n");
    printf(" 0 | 1  2  3 | 4  5  6 | 7  8  9
| X\n");
    printf(" ---------------------------------
-\n");
    for (i = 0, a = 1; i < 9; i++, a++){
        for(j = 0; j < 9; j++){
            if (j == 0)
                printf(" %d |", a);
            else if ((j) % 3 == 0)
                printf("|");
            printf(" %d ", puzzle[i][j]);
            if (j == 8)
                printf("|");
        }
        printf("\n");
        if ((i + 1) % 3 == 0)
            printf(" ------------------------
---------\n");
    }
    printf(" Y\n");
}

void userChoice(int** userPuzzle, int**
tempPuzzle){
    int i, j;
    int positionX, positionY, userVal;
    char c;

    while(1){
    if(!checkAvailable(userPuzzle, &i, &j)){
        printf("\nPuzzle is solved.\n");
        return;
    }

    while(1){
        printf("\nPress Enter to continue.
\n");
        c = getchar();
        if((c == 'q') || (c == 'Q')){
            getchar();

            return;
        }
        else if((c != '\n') && (c != 'q'))
            getchar();
        else
            break;
    }


    printf("\n Enter Coordinate  in 'X Y'
format :\n");
    scanf("%d %d",&positionX, &positionY);
    while(1){
        if ((positionX > 9) || (positionX <
1) || (positionY > 9) || (positionY < 1) ||
(userPuzzle[positionY - 1][positionX - 1] !=
0)){
 printf("\nPlease insert value from 1 to
9\n");
    scanf("%d", &userVal);
    while(1){
        if((userVal > 9) || (userVal < 1)){
            printf("\nplease insert valid
value:\n");
            scanf("%d", &userVal);
        }
        else break;
    }

    if(checkBox(userPuzzle, positionY,
positionX, userVal))
        userPuzzle[positionY][positionX] =
userVal;
    else
        printf("\nincorrect value for the X =
%d Y = %d coordinate, please try
again\n",positionX + 1, positionY + 1);

    for (i = 0; i < 9; i++){
        for(j = 0; j < 9; j++){
            tempPuzzle[i][j] =
userPuzzle[i][j];
        }
    }

    if(!solvePuzzle(tempPuzzle)){
        printf("\nincorrect value for the X =
%d Y = %d coordinate, please try
again\n",positionX + 1, positionY + 1);
        userPuzzle[positionY][positionX] = 0;
    }
    getchar();
    printPuzzle(userPuzzle);
    }

    return;

bool checkAvailable(int** puzzle, int* row,
int* column){
    int i, j;

    for (i = 0; i < 9; i++){
        for(j = 0; j < 9; j++){
            if (puzzle[i][j] == 0){
                *row = i;
                *column  = j;
                return true;
            }
        }
    }
```

Code Link:

https://drive.google.com/file/d/1TfLGzyFOZDok_5CwENhGbT3Ttqj99Xyt/view?usp=sharing